

Ministerul Educației și Cercetării
Palatul Copiilor Drobeta Turnu Severin

Programă de opțional

PYTHON I (UCENIC)
LOGICĂ SECVENȚIALĂ ȘI INTRODUCERE ÎN PROGRAMARE

Nivelul: începători¹

Autor
prof. ing. Mihai Agape

Drobeta Turnu Severin
2026

¹ Nivelul **începători** corespunde nivelului grupelor din cadrul cercului, cărora li se adresează programa.

Notă de prezentare

Cadrul curricular general

Predarea programării în învățământul preuniversitar a fost tradițional dominată de informatica teoretică, creând adesea bariere cognitive timpurii din cauza sintaxelor abstracte. Prezentul curriculum propune o schimbare de paradigmă, migrând către un focus pe **informatică aplicată și procesare a datelor**, în deplină concordanță cu recomandările europene privind competențele digitale cheie. Inspirat din modelul pedagogic universitar de succes dezvoltat de Dr. Charles Severance (*Python for Everybody*), scopul de bază este de a dota elevii cu **abilități practice de manipulare, analiză și automatizare a datelor pe tot parcursul vieții**, utile în orice domeniu de activitate. Limbajul **Python 3** este vehiculul tehnologic unic ideal datorită sintaxei curate, extrem de apropiate de pseudocodul în limba engleză, a sistemului de execuție interpretat cu feedback vizual instantaneu și a portabilității depline.

Arhitectura pe 5 niveluri este structurată sub forma unei bresle profesionale în care elevii progresează în propriul ritm, obținând grade de „senioritate” pe măsură ce își finalizează proiectele în mod autonom:

1. **Nivelul I: Ucenic** (*Python I: Logică secvențială și introducere în programare*) — se concentrează pe fundamentele fizice ale calculatorului, ierarhia de foldere și fișiere, decompoziția sarcinilor cotidiene și scrierea primelor scripturi secvențiale liniare.
2. **Nivelul II: Junior** (*Python II: Logica controlului de flux și grafică cu Turtle*) — introduce deciziile condiționale, buclele repetitive, interacțiunea dinamică și biblioteca grafică Turtle.
3. **Nivelul III: Senior** (*Python III: Structuri de date și modularizare*) — se concentrează pe colecții de date (liste, dicționare), persistența datelor în fișiere text și de tip CSV, modularizare în funcții și aserțiuni de autoevaluare.
4. **Nivelul IV: Expert** (*Python IV: Stăpânirea Python, programare funcțională și bazele OOP*) — abordează concepte avansate de programare funcțională, recursivitate și programarea orientată pe obiecte (OOP), completate de gestiunea proiectelor în echipă.
5. **Nivelul V: Maestru** (*Python V: Sisteme complexe, Web și baze de date relaționale*) — reprezintă anul de sinteză dedicat bazelor de date SQL (limbaj de interogare structurat - eng. *Structured Query Language*), API-urilor (interfețe de programare a aplicațiilor - eng. *Application Programming Interface*), securității software defensive, framework-ului web Flask și versionării codului prin Git/GitHub.

Pentru a asigura o învățare profundă și incluzivă în mediul nonformal, întregul curriculum se bazează pe patru piloni metodologici moderni. Primul este **gândirea computațională** (eng. *computational thinking*) ca motor de învățare, structurată pe descompunerea problemelor (decompoziție), identificarea tiparelor (recunoașterea de tipare), abstracție și proiectarea algoritmilor. Al doilea pilon este normalizarea **efortului productiv în depășirea blocajelor** (eng. *productive struggle*) prin predarea explicită a **depanării** (eng. *debugging*), abordată ca proces sistematic bazat pe interpretarea mesajelor de eroare și urmărirea stărilor din memorie. Al treilea pilon este pedagogia **caietelor interactive de lucru** (eng. *interactive notebooks* sau *Jupyter Notebooks*) găzduite în cloud, o abordare inspirată de pe principiul **programării explicate** (eng. *literate programming*), care îmbină explicațiile teoretice cu celule de cod executabile în timp real. Al patrulea pilon este **designul universal pentru învățare** (eng. *Universal Design for Learning* - UDL) care folosește multiple mijloace de

reprezentare (vizuale, textuale, kinestezice) și **activități fără calculator** (eng. *unplugged*), sprijinind trecerea lină de la limbaje vizuale (ex: Scratch) la cod text.

Dincolo de competențele tehnice, elevii își formează o cultură a dezvoltării responsabile de software, bazată pe planificare, organizarea etapelor de lucru, colaborare în perechi (eng. *pair programming*) și revizuirea colegială a codului (eng. *code review*). Activitățile încurajează utilizarea unor metode agile de planificare, atât pe hârtie, cât și cu ajutorul panourilor digitale, precum și oferirea de feedback constructiv între colegi. Totodată, elevii sunt familiarizați cu principiile eticii digitale, cu respectarea licențelor software și cu recunoașterea contribuției comunității open-source.

Notă de prezentare pentru nivelul I (ucenic)

Prezentă programă este concepută pentru disciplina **Python I (Ucenic): Logică secvențială și introducere în programare**, reprezentând primul nivel de studiu din cadrul curriculumului unificat de *Informatică aplicată cu Python*. Cursul se adresează deopotrivă fetelor și băieților care nu au nicio experiență anterioară în scrierea de cod pe calculator, ajutându-i să se transforme din consumatori pasivi de tehnologie în creatori activi ai acesteia. Cursul se va desfășura pe durata unui an școlar, în cadrul unor activități săptămânale cu durata de 100 minute (sau 2 ore didactice de 50 de minute pe săptămână).

Scopul principal al cursului de Python I (nivelul Ucenic) este alfabetizarea digitală și inițierea elevilor în gândirea algoritmică și scrierea primelor programe textuale. Aceasta se face prin activități preponderent aplicative și practice care mențin interesul elevilor pe parcursul cursului. Activitățile desfășurate constituie o oportunitate pentru exersarea de către elevi a competențelor dobândite la diverse discipline din trunchiul comun (matematică, tehnologii, limbi străine, TIC, fizică, chimie, etc.), ceea ce duce la întărirea motivației pentru studiul acestora.

Trăim într-o lume puternic tehnologizată, în care programarea își găsește exprimarea în aproape toate lucrurile care ne înconjoară. Este normal ca elevii să-și dezvolte cunoștințe și deprinderi în acest domeniu. Acest fapt rezultă și din documentele Comisiei Europene, care stabilesc că unele dintre competențele cheie pe care trebuie să și le dezvolte elevii sunt cele digitale și de bază în științe și tehnologii. Chiar dacă acest argument se bucură de sprijinul autorității, el nu este suficient pentru atragerea elevilor către un curs de programare. De aceea cel de-al doilea argument se referă la beneficiile pe care le are elevul prin participarea la un curs de programare și care nu se reduc la cunoștințe și deprinderi specifice domeniului informaticii.

Activitățile specifice programării constituie un prilej excelent pentru formarea și dezvoltarea unor competențe care transcend domeniul strict informatic și sunt utile pe parcursul întregii vieți. Activitățile desfășurate la cursul de Python I sprijină atât formarea și/sau dezvoltarea unor competențe specifice domeniului (*hard skills*), cât și dezvoltarea competențelor interpersonale (*soft skills*). De exemplu, pentru realizarea unui proiect, o echipă de elevi parcurge următoarele etape:

- identificarea unor probleme din jurul lor;
- planificarea proiectului;
- documentarea pe marginea temei (folosind Internetul);
- proiectarea algoritmului pe hârtie prin pseudocod sau diagrame de flux;
- scrierea, testarea și depanarea codului în caiete interactive de lucru;
- realizarea unei documentații simple;

- prezentarea proiectului live în fața unui auditoriu.

Din cele de mai sus rezultă că un curs de informatică aplicată contribuie și la dezvoltarea altor competențe cheie precum: comunicarea în limba maternă și în limbi străine, competențe matematice, digitale, sociale, antreprenoriale și competența de a învăța să înveți. Pe parcursul activităților sunt antrenate capacități precum creativitatea, gândirea critică, rezolvarea de probleme, dar și o atitudine pozitivă față de lucrul bine făcut și lucrul în echipă.

Disciplina de studiu se concentrează pe o logică secvențială (execuția instrucțiunilor una după alta, în ordinea scrierii lor, de sus în jos), fiind ghidată de principiul pedagogic „o singură idee majoră pe sesiune”. Statutul de **Ucenic** oferă elevului o identitate activă plină de demnitate în cadrul breslei de programatori, normalizând greșeala sintactică drept o etapă firească a învățării, ghidată de un meșter formator (profesorul). Cursul clădește un model mental solid privind funcționarea mașinii fizice (CPU, RAM, stocare permanentă), o disciplină tehnică riguroasă de management al fișierelor și deprinderea de tehnici de depanare conștientă (urmărirea stărilor în memorie prin afișări intermediare).

Programa este structurată similar programelor pentru învățământul formal, cuprinzând: notă de prezentare, competențe generale, competențe specifice cu exemple de activități de învățare asociate, conținuturi, sugestii metodologice și de evaluare.

- **Competențele generale** au un grad ridicat de generalitate și sunt competențe formate pe parcursul celor 5 niveluri de studiu ale disciplinei Informatică aplicată cu Python.
- **Competențele specifice** sunt derivate din competențele generale și sunt diferite pentru fiecare nivel de studiu.
- **Activitățile de învățare** reprezintă exemple de sarcini de lucru prin care se formează și se dezvoltă competențele specifice.
- **Conținuturile** sunt mijloace informaționale care sprijină formarea competențelor.
- **Sugestiile metodologice și de evaluare** cuprind recomandări generale, care au rolul de a oferi profesorului exemple referitoare la strategiile didactice și resursele necesare care facilitează dezvoltarea competențelor.

Competențele generale

CG 1. Gestionează mediul de lucru și datele necesare dezvoltării aplicațiilor Python, incluzând organizarea fișierelor, directoarelor și formatelor de date.

CG 2. Aplică gândirea computațională pentru descompunerea problemelor, identificarea tiparelor, abstracție și proiectarea algoritmilor.

CG 3. Dezvoltă programe în Python respectând sintaxa limbajului și convențiile de stil PEP 8.

CG 4. Verifică robustețea programelor Python prin validarea datelor de intrare, testarea pe parcursul dezvoltării, interpretarea mesajelor de eroare și activități de depanare.

CG 5. Coordonează proiecte software prin planificare, documentare, colaborare și respectarea principiilor de etică digitală.

Competențe specifice și exemple de activități de învățare

CG 1. Gestionează mediul de lucru și datele necesare dezvoltării aplicațiilor Python, incluzând organizarea fișierelor, directoarelor și formatelor de date.

CS 1.01. . Identifică rolul componentelor hardware de bază, al sistemului de operare și al interpretorului Python în rularea aplicațiilor.

- Joc de rol: „Componentele în acțiune” (elevii joacă rolul CPU, RAM și stocare pentru a simula execuția unui program).
- Fișă de lucru interactivă bazată pe analogia unei case, pentru asocierea memoriei RAM și a stocării permanente cu rolurile lor.
- Discuție ghidată despre rolul sistemului de operare, pornind de la analiza consecințelor lipsei acestuia.
- Executarea unor instrucțiuni simple în interpretorul interactiv și observarea rezultatelor produse.
- Crearea și configurarea unui notebook nou în Google Colab, denumit Prima_Mea_Lectie.

CS 1.02. Organizează fișierele și folderele de proiect într-o structură logică de directoare

- Exercițiu practic de organizare a unui spațiu de lucru: crearea structurii de foldere CursPython/Modulul1/Teme.
- Compararea modului de stocare și afișare a informației într-un fișier .txt și într-un document .docx.
- Laborator „Vânătoarea de extensii”: identificarea și sortarea fișierelor de pe un stick USB după extensie (.py, .docx, .png, .mp3).
- Redenumirea fișierului „tema mea 1.py” utilizând o denumire clară, consecventă și fără spații, de exemplu „tema_mea_1.py”.
- Exportarea notebookului din Google Colab în formatele .ipynb și .py, apoi localizarea și deschiderea fișierelor rezultate din folderul local.

CG 2. Aplică gândirea computațională pentru descompunerea problemelor, identificarea tiparelor, abstracție și proiectarea algoritmilor.

CS 2.01. Descompune procese și activități cotidiene în pași logici secvențiali ordonați.

- Decompoziția sarcinilor: Elevii scriu pe cartonașe separate fiecare micro-pas necesar pentru prepararea unui ceai.
- Ordonarea cartonașelor și verificarea succesiunii pașilor prin simularea preparării ceaiului.
- Analiza unor variante de preparare a ceaiului în care pași sunt inversați, omiși sau repetați; elevii explică efectul fiecărei modificări asupra rezultatului.
- Proiectarea și trasarea, pe hârtie milimetrică, a unei succesiuni de comenzi pentru desenarea unui pătrat, indicând distanțele și virajele.
- Asocierea unei structuri repetitive din Blockly/Scratch cu repetarea controlată a unei acțiuni din viața reală.

CS 2.02. Reprezintă algoritmi liniari simpli prin pseudocod și diagrame de flux.

- Dezbatere ghidată privind caracteristicile unui algoritm, pornind de la analiza unei rețete culinare (ex: „Este o rețetă de cozonac un algoritm?”). Identificarea caracteristicilor rețetei.
- Scrierea în pseudocod a algoritmului pentru calculul mediei a două note, precizând datele de intrare, prelucrările și rezultatul afișat (ex: citește nota1, citește nota2, calculează suma...).
- Construirea unei diagrame de flux pentru adunarea a două numere, folosind simbolurile Start/Stop, intrare/ieșire și proces.
- Asocierea operațiilor „citește” și „afișează” cu simbolul corespunzător de intrare/ieșire dintr-o diagramă de flux.
- Simularea manuală a execuției unei diagrame de flux cu valori de test și completarea unui tabel de stări pentru variabile.

CG 3. Dezvoltă programe în Python respectând sintaxa limbajului și convențiile de stil PEP 8.

CS 3.01. Afișează texte și formatează ieșiri în consolă utilizând funcția print și caractere escape.

- Exercițiul clasic „Hello World”: Elevii scriu și rulează instrucțiunea `print('Hello World!')` în Colab.
- Scrierea unor mesaje care includ ghilimele sau apostrofe, alegând corect delimitatorii și Țirului de caractere. Exemplu: `print('El a spus: "Bună!")`.
- Afișarea unui text pe mai multe rânduri Țintr-o singură instrucȚiune `print()`, utilizȚnd caracterul `\n`.
- Afișarea Țn consolă a unui tabel simplu cu nume și note, utilizȚnd caracterul `\t` pentru separarea coloanelor.
- Scrierea și compararea mai multor apeluri `print()` cu argumente multiple, pentru observarea separatorului implicit și a modului de afișare.

CS 3.02. Declară variabile pentru stocarea și utilizarea tipurilor primitive de date.

- Crearea și denumirea unor variabile pentru nume, vȚrstă și ȚnȚlȚime, respectȚnd convenȚia `snake_case`.
- Identificarea și corectarea unei atribuirii invalide, de exemplu transformarea lui `12 = varsta` Țn `varsta = 12`.
- Utilizarea funcȚiei `type()` pentru verificarea tipului unor valori literale și al unor variabile declarate.
- Modificarea valorii unei variabile prin reatribuire și afișarea valorii Țnainte și dupȚ actualizare (ex: variabila `pret` are ȚnȚial valoarea 100 și apoi devine 80 dupȚ aplicarea unei reduceri).

- Compararea afișării valorii variabilei x prin print(x) cu afișarea șirului literal 'x' prin print('x').

CS 3.03. Procesează datele introduse interactiv de utilizator prin aplicarea operatorilor aritmetici.

- Crearea unui chestionar interactiv cu trei întrebări și afișarea răspunsurilor introduse de utilizator (ex: „Care este animalul tău preferat?”).
- Realizarea unui mini-calculator care citește două valori, le convertește cu int() sau float() și calculează suma lor, prevenind erorile de tip.
- Scrierea unui script pentru calculul ariei unui dreptunghi utilizând dimensiunile oferite interactiv de la tastatură.
- Crearea unui program care citește separat numele și prenumele, apoi afișează un mesaj personalizat: „Salut ” + prenume + " " + nume.
- Calcularea manuală a rezultatului unei expresii, formularea unei predicții și verificarea acesteia prin rularea expresiei în Python (ex: rezultat = 5 + 3 * 2).

CG 4. Verifică robustețea programelor Python prin validarea datelor de intrare, testarea pe parcursul dezvoltării, interpretarea mesajelor de eroare și activități de depanare.

CS 4.01. Corectează erorile elementare de sintaxă apărute în execuția codului.

- Introducerea unei erori de sintaxă controlate (ex: ghilimele neînchise), observarea mesajului SyntaxError și formularea unei ipoteze despre cauza ei.
- Elevii analizează fragmente de cod cu erori, localizează linia indicată de interpretor, formulează o ipoteză despre cauză și verifică ipoteza prin corectarea și rerularea codului.
- Completarea parantezelor lipsă în fragmente scurte de cod și verificarea sintaxei prin rulare (ex: sarcina „Închideți colivia”).
- Analizarea, clasificarea și corectarea unor fragmente de cod cu delimitatori de text lipsă sau nepotrivți, urmate de rerularea programelor.
- În perechi, elevii identifică erori de tastare în fragmente scurte de cod, le corectează, rulează programul și explică modul în care mesajul de eroare i-a ajutat.

CS 4.02. Urmărește execuția unui program liniar prin tehnica afișărilor intermediare (eng. print tracing).

- Înainte de rulare, elevii prevăd valorile variabilelor după fiecare instrucțiune; apoi rulează codul și compară predicțiile cu stările observate.
- Inserarea de instrucțiuni print() temporare pentru observarea valorilor intermediare ale variabilelor și localizarea unui rezultat neașteptat.
- Completarea unui tabel cu valori așteptate și valori obținute, în paralel cu rularea codului, pentru identificarea momentului în care apare diferența.
- Depanarea unui mini-program de calcul al vârstei peste un an prin formularea unei predicții, compararea rezultatului obținut cu cel așteptat și izolarea instrucțiunii care produce diferența.
- Revizuirea codului după depanare: elevii elimină instrucțiunile temporare de urmărire, verifică dacă programul funcționează în continuare și adaugă comentarii numai unde acestea clarifică intenția codului.

CG 5. Coordonează proiecte software prin planificare, documentare, colaborare și respectarea principiilor de etică digitală.

CS 5.01. Planifică cerințele unui mini-proiect personal prin schițarea în echipă a algoritmului.

- În perechi, elevii analizează cerința unui mini-proiect, formulează întrebări de clarificare și completează o fișă cu intrări, prelucrări, ieșiri și exemple de rezultate așteptate (ex: program de calcul al vârstei de pensionare).

- Elevii creează pe hârtie un storyboard al interacțiunii: mesaj de întâmpinare, date de intrare, prelucrare și rezultat afișat; apoi explică traseul utilizatorului prin aplicație.
- Echipa stabilește roluri, sarcini, termen estimat și criterii de finalizare pentru fiecare responsabilitate; la finalul activității, membrii verifică dacă distribuția este echilibrată și o ajustează dacă este necesar.
- Fiecare membru propune o soluție, echipa stabilește criterii de comparație — corectitudine, claritate și număr de pași — apoi argumentează alegerea sau combinarea soluțiilor.
- În echipă, elevii compară algoritmul desenat cu execuția programului, consemnează o concordanță și o diferență, apoi revizuiesc schema sau codul pentru a obține coerență.

CS 5.02. Prezintă funcționalitatea unui program propriu prin demonstrații live și comunicare tehnică.

- Elevii redactează un enunț despre scopul aplicației, îl verifică în perechi cu ajutorul unor criterii de claritate și îl rescriu înainte de o prezentare scurtă.
- În grupuri mici, elevii proiectează cazuri de test cu valori obișnuite, valori-limită și date necorespunzătoare, prezic rezultatele, rulează programul și discută eventualele diferențe.
- În perechi, elevii etichetează într-un fragment de cod elementele: variabilă, șir literal, intrare și ieșire; apoi își revizuiesc descrierea propriului program folosind termenii potriviți.
- În perechi, elevii formulează întrebări despre datele de intrare, tipurile de date și rezultatele programului (ex: „De ce ai folosit tipul float și nu int pentru valoarea banilor?”). Răspunsurile urmează structura „Am ales..., deoarece...”, apoi sunt îmbunătățite după feedback.
- În perechi, elevii utilizează o grilă cu criterii: funcționare, claritatea codului, organizarea și comunicarea rezultatului. Oferă un aspect reușit, o întrebare și o sugestie concretă; autorul selectează o sugestie pe care o poate aplica.

Conținuturi

Domenii de conținut	Conținuturi
Arhitectura de bază și medii de lucru	- Componente fizice de bază - RAM vs. stocare - Sistemul de operare - Interpretorul Python - Medii de lucru în cloud
Managementul fișierelor	- Ierarhia de directoare - Tipuri de fișiere (.txt vs. .docx) - Extensia standard .py - Reguli de numire a fișierelor - Exportarea proiectului în formatele .ipynb și .py
Descompunerea sarcinilor (Decompoziție)	- Identificarea micro-pașilor - Ordonarea secvențială - Erori logice în ordonarea pașilor - Decompoziție grafică - Corespondența cu Blockly/Scratch

Domenii de conținut	Conținuturi
Pseudocod și Diagrame de flux	- Conceptul de algoritm - Scrierea în pseudocod - Simboluri diagramă de flux (Proces) - Simboluri diagramă de flux (I/O) - Simularea manuală a execuției (dry run)
Afișarea datelor și caractere escape	- Apelarea funcției print() - Delimitatori text (ghilimele) - Caracter escape \n (linie nouă) - Caracter escape \t (tabulare) - Afișarea itemilor multipli
Variabile și tipuri primitive de date	- Numirea variabilelor (snake_case) - Sintaxa de atribuire (=) - Tipuri de date primitive (type) - Reatribuire și starea memoriei - Afișarea variabilelor în consolă
Preluarea intrărilor și calcule aritmetice	- Preluarea de text cu input() - Conversia de tip (int, float) - Operatori aritmetici fundamentali - Concatenarea șirurilor de caractere - Precedența operatorilor matematici
Tehnica afișărilor intermediare (print tracing)	- Logica de sus în jos (secvențialitate) - Afișarea intermediară a stării - Valori teoretice vs. valori reale - Izolarea liniei defecte - Curățarea codului după depanare
Depanarea erorilor de sintaxă	- Identificarea SyntaxError - Localizarea și interpretarea erorilor - Corectarea parantezelor neînchise - Corectarea ghilimelelor nepotrivite - Corectarea erorilor de tastare
Planificarea mini-proiectelor în echipă	- Cerințe funcționale: intrări, prelucrări și ieșiri - Proiectarea interacțiunii și a algoritmului - Planificarea și distribuirea sarcinilor - Compararea și negocierea soluțiilor - Compararea planului cu rezultatul
Prezentarea orală și demonstrația live	- Formularea scopului programului - Testarea programului cu cazuri de intrare - Terminologie tehnică aplicată - Argumentarea alegerilor de programare - Feedback pentru îmbunătățirea produsului

Sugestii metodologice

Implementarea curriculumului **Python I (Ucenic): Logică secvențială și introducere în programare** în mediul nonformal necesită strategii didactice interactive, profund practice și centrate pe elev, capabile să mențină o înaltă implicare emoțională și cognitivă, evitând abandonul cauzat de frustrările sintactice.

Proiectarea didactică se va realiza astfel încât dobândirea competențelor să se realizeze gradat, corespunzător diferitelor niveluri ale competenței, urmărind ca fiecare nivel al competenței să fie mai complex decât precedentul și achizițiile unuia să fie implicate în următorul. Introducerea conținuturilor se va face, pe cât posibil, în momentul existenței unei probleme practice la a cărei soluționare contribuie.

Rămâne la decizia profesorului formator modul de eșalonare a activităților, astfel încât parcurgerea programei să fie eficientă și atractivă. În continuare sunt prezentate succint câteva sugestii referitoare la strategiile didactice, activitățile de învățare, modalitățile de evaluare, mijloace și conținuturi.

Strategii didactice

Pentru dezvoltarea competențelor la elevi, în procesul de predare-învățare-evaluare, se vor folosi metode activ-participative precum: observația, conversația euristică, descoperirea dirijată, problematizarea, experimentul, lucrarea practică, studiul individual, brainstormingul, proiectul și programarea în perechi (eng. *pair programming*).

Elevii vor fi puși în diferite situații de învățare în care vor rezolva sarcinile de lucru în grupe de elevi sau echipe, pentru a dezvolta spiritul de colaborare, comunicarea interactivă și abilitățile interpersonale (*soft skills*).

Activitățile de învățare vor fi centrate pe elev și vor avea un caracter aplicativ și practic. Elevii vor fi antrenați în rezolvarea unor probleme de programare, crearea de scenarii grafice pas cu pas (eng. *storyboard*), crearea, testarea, depanarea și evaluarea programelor scrise de ei înșiși și de colegi. Activitățile de învățare vor fi adaptate în funcție de soluțiile oferite de elevi și de problemele apărute pe parcursul activităților.

În mod specific pentru nivelul I (Ucenic), se recomandă aplicarea următoarelor strategii și tehnici didactice:

Strategia hibridă (deconectat, vizual, textual)

Pentru a asigura o ghidare didactică progresivă (eng. *scaffolding*) eficientă, fiecare concept major este introdus prin trei etape succesive.

Etapa fără calculator (eng. unplugged)

Conceptele abstracte sunt explorate mai întâi în planul fizic, kinestezic. De exemplu, înainte de a preda variabilele și stările memoriei, elevii realizează jocuri de rol (cum ar fi „Componentele în acțiune” sau manipularea unor cutii fizice etichetate pentru a ilustra reatribuirea variabilelor). Algoritmizarea este deprinsă utilizând cartonașe fizice colorate pentru a ordona micro-pașii preparării unui ceai sau pentru a simula comportamentul unui robot.

Etapa vizuală (tranziția de la programare bazată pe blocuri)

Se valorifică intuiția formată în limbajele vizuale bazate pe blocuri. Elevii sunt ghidați să asocieze direct blocurile logice pe care le cunosc (afișări, atribuirii de variabile) cu echivalentul lor sub formă de text în Python, facilitând depășirea blocajelor cognitive.

Etapa textuală

Elevii transpun algoritmi în cod real Python, scriind instrucțiuni clare direct în Google Colab sau în caiete interactive de lucru (Jupyter Notebooks).

Metodologia „Meșterește, ajustează și observă” (eng. Tweak, Twiddle, and Frob)

Pentru a înlătura frica ecranului gol la începători, se utilizează bunele practici de design instrucțional pentru caietele interactive de lucru. În loc să fie puși să scrie programe lungi de la zero, elevii primesc caiete interactive de lucru pre-populate, care conțin explicații conceptuale clare (text formatat) și celule de cod complet funcționale. Elevii aplică tehnica:

- **Rulează** celula de cod existentă și observă rezultatul în timp real.
- **Modifică** parametrii, valorile variabilelor, mesajele text sau separatorii (ex. schimbă \n cu \t sau modifică valoarea unei variabile de preț de la 100 la 80).
- **Rulează** din nou codul după modificare pentru a observa direct efectul asupra rezultatului în consolă.
- **Trag concluzii logice** privind modul în care modificările afectează ieșirea programului.

Abordarea erorii ca resursă didactică (depanare explicită)

Mesajele roșii de eroare sunt transformate dintr-o formă de penalizare într-un instrument de investigație științifică. Profesorul nu va corecta niciodată erorile de pe ecranul elevilor; în schimb, îi va ghida să aplice o metodologie de depanare explicită:

- **Sarcina „Închide colivia”** pentru corectarea parantezelor neînchise în apelurile de funcții imbricate (cum ar fi `print(int(input()))` unde elevii uită frecvent ultima paranteză), se folosește metafora vizuală a coliviei. Fiecare paranteză deschisă (reprezintă o pasăre care va zbura dacă nu închidem colivia la final cu o paranteză pereche).
- **Simularea la rece (pe hârtie) și urmărirea stărilor prin afișări intermediare** în cazul în care un algoritm scoate un rezultat eronat. Elevii sunt opriți de la modificările haotice din „ghicite”. Ei sunt învățați să completeze un tabel de stări pe hârtie (eng. *dry run*), calculând manual ce ar trebui să fie în memorie, și să insereze instrucțiuni `print()` intermediare temporare pentru a monitoriza valorile reale ale variabilelor, identificând precis linia care corupe datele.

Modele și exemple ancorate în realitate

Toate exemplele practice folosite la clasă sunt calibrate pe nivelul de interes și de înțelegere al elevilor, evitându-se problemele pur matematice complicate în favoarea unor contexte distractive:

- **Tabel de note personalizat** pentru afișarea în consolă a numelui și a notelor obținute la materii utilizând caracterele \t pentru separarea elementelor în coloane.
- **Mini-calculator interactiv** pentru calcularea ariei unui dreptunghi sau a sumei a două numere pe baza dimensiunilor introduse live de utilizator de la tastatură, cu obligativitatea conversiei tipului primit de funcția `input()`.
- **Simularea aplicării unei reduceri** a prețului unui produs la magazin (inițial 100 de lei, reatribuit la 80 de lei după aplicarea reducerii de 20%).

Sugestii de evaluare și autoevaluare

În mediul nonformal, evaluarea renunță complet la caracterul său punitiv sau de sortare (clasicele note și extemporale stresante), transformându-se într-un proces continuu, formativ și profund autonom, menit să sprijine progresul fiecărui ucenic.

Se vor aplica toate tipurile de evaluare: inițială, formativă și sumativă. Cel mai mult va fi folosită evaluarea formativă, care permite profesorului să-și ajusteze activitatea pentru a sprijini formarea competențelor la elevi. Profesorul îi va încuraja pe elevi să se autoevalueze și să se evalueze unul pe celălalt în vederea îmbunătățirii modului în care îndeplinesc anumite activități.

Ca instrumente de evaluare se vor utiliza: observarea sistematică a activității, teste formative, probe practice, lucrări practice pentru depistarea și înlăturarea unui defect (depanare), evaluare colegială, proiectul și portofoliul.

Evaluarea formativă automată (prin aserțiuni de autoverificare automată)

Pentru a încuraja autonomia ucenicului și pentru a elimina dependența permanentă de confirmarea profesorului, în caietele interactive de lucru sunt integrate celule de verificare automată folosind instrucțiunea de aserțiune și verificare (assert).

Cum funcționează

Sub celula de lucru în care elevul trebuie să rezolve o sarcină (de exemplu, să calculeze aria unui dreptunghi în variabila `arie_dreptunghi`), se află o celulă de test ascunsă sau blocată, conținând verificarea (aserțiunea): `assert arie_dreptunghi == 50, "Aria calculată nu este corectă! Verifică dacă ai înmulțit corect lungimea cu lățimea!"`

Avantaj didactic

Elevul rulează celula de test. Dacă testul are succes, celula rulează în liniște, confirmându-i vizual succesul și oferindu-i undă verde de a trece singur la exercițiul următor. Dacă testul eșuează, verificarea aruncă un mesaj personalizat explicativ, ajutându-l să înțeleagă unde a greșit și stimulând efortul productiv în depășirea blocajelor fără a apela la profesor.

Observarea sistematică a activității

Profesorul urmărește sistematic comportamentul elevului în timpul laboratoarelor, utilizând o fișă de observare pentru a urmări calitatea deprinderilor și a atitudinilor:

- Respectarea convențiilor de numire `snake_case` și a standardului PEP 8.
- Abilitatea de a citi independent mesajul de eroare `SyntaxError` înainte de a cere asistență.
- Organizarea structurată a folderelor de proiect pe disc.
- Capacitatea de a oferi sprijin colegilor aflați în dificultate.

Evaluarea colegială prin tehnica „Ce a strălucit & Cum putem crește” (eng. glow & grow)

La finalul fiecărui capitol major sau mini-proiect realizat în perechi, elevii își prezintă live programul în fața clasei, rulându-l cu valori de test oferite la întâmplare de către public. Evaluarea realizată de colegi este guvernată de o tehnică constructivă de **evaluare colegială reciprocă** (eng. *peer review*).

Ce a strălucit (eng. glow)

Fiecare evaluator colegial identifică și laudă un aspect excelent implementat în proiect (de exemplu, un mesaj de întâmpinare foarte prietenos, o organizare vizuală impecabilă cu caractere `escape \t` sau o logică de calcul corectă).

Cum putem crește (eng. grow)

Evaluatorul oferă o singură sugestie constructivă, formulată politicos, pentru o optimizare viitoare (de exemplu, redenumirea unei variabile conform PEP 8 sau adăugarea unui comentariu de tip `#` pentru a clarifica o formulă matematică).

Această practică îi deprinde pe elevi cu cultura de revizuire a codului din industria reală a tehnologiei, eliminând frustrarea criticii și dezvoltând empatia, ascultarea activă și colaborarea academică sănătoasă.

Portofoliul digital

Fiecare elev își assemblează pe parcursul celor 36 de săptămâni un portofoliu digital în Google Drive sau GitHub, conținând:

- Diagramele de flux și schițele scenariilor grafice pas cu pas realizate pentru mini-proiecte.
- Fișierele de cod sursă `.py` și caietele interactive de lucru `.ipynb` exportate local din cloud.
- Prezentarea finală a proiectului de sinteză (eng. *capstone project*).

Acest portofoliu reprezintă dovada palpabilă a saltului calitativ realizat, servind ca fundament pentru evaluarea finală de trecere la Nivelul II (Junior) și oferindu-i elevului o imensă mândrie personală.

Mijloace și materiale didactice

Infrastructură hardware

- Calculatoare sau laptopuri conectate la internet (recomandat un echipament per elev sau minimum un echipament la doi elevi pentru lucrul în perechi).
- Videoproiector sau tablă interactivă pentru prezentarea frontală a schemelor și demonstrațiilor.

Medii de dezvoltare și instrumente online

- Caiete interactive de lucru (fișiere în format .ipynb) găzduite la distanță pe platformele Google Colab sau Jupyter Notebooks.
- Platforma de vizualizare în timp real a memoriei și a fluxului de execuție a codului **Python Tutor**.
- Editoare online și console interactive optimizate pentru dispozitive diverse.
- Platforma de management al învățării Moodle ca spațiu de găzduire a caietelor interactive de lucru, oferind ucenicilor un punct centralizat de unde pot accesa și descărca direct fișierele .ipynb necesare laboratoarelor practice și evaluărilor sumative.

Suport de curs și culegeri de resurse

- Acces la lecții online interactive în limba română (de tipul platformei *pythonisti.ro*).
- Cărți și manuale în format electronic, cum ar fi *Python 3 - Fundamente pentru începători* (de Vlad Tudor) sau manualul ilustrat *Super Aventura Python pe Marte!*.

Materiale fizice pentru activități fără calculator

- Seturi de cartonașe fizice colorate pentru exercițiile de decompoziție și ordonare secvențială a algoritmilor cotidieni.
- Fișe de lucru tipărite cu tabele de stări (pentru simularea la rece pe hârtie a diagramelor de flux).
- Carduri fizice de corespondență vizuală între blocurile logice din Scratch și instrucțiunile textuale din Python.

Bibliografie

- [1] **Agape, M.** , *Programă de opțional: Electronică* , Palatul Copiilor Drobeta Turnu Severin, 2026.
- [2] **Agape, M.** , *Programă de opțional: Informatică cuantică - novice* , Palatul Copiilor Drobeta Turnu Severin, 2026.
- [3] **Agape, M.** , *Programă de opțional: Programare în Scratch* , Palatul Copiilor Drobeta Turnu Severin, 2026.
- [4] **Agape, M.** , *Programă de opțional: Robotică* , Palatul Copiilor Drobeta Turnu Severin, 2026.

- [5] **Al-Zubaidy, H. et al.** , *An Investigation Into Secondary School Students' Debugging Behaviour in Python* , arXiv preprint, 2024.
- [6] **Atlantis Press** , *Jupyter Notebooks as Computational Thinking Tools for Teaching and Lifelong Learning in Biotechnology* , 2022.
- [7] **Code for Schools** , *Introduction to Turtle Graphics!* , codeforschools.co.uk.
- [8] **ETH Pressbooks** , *Computational Competencies: Using JupyterHub with Moodle* , ETH Zürich, 2023.
- [9] **Kaila, E. et al.** , *Teaching Python programming with automatic assessment and feedback provision* , arXiv preprint, 2018.
- [10] **Severance, C.** , *Python for Everybody: Exploring Data In Python 3* , dr-chuck.com, 2016.